



Directory > ... >

Network-focused migration from VPN
concentrators to Zero Trust Network
Access

Copy page



Network-focused migration from VPN concentrators to Zero Trust Network Access

Last reviewed: about 1 year ago

Introduction

Over the past few years, the traditional approach of installing and maintaining hardware for remote access to private company networks is no longer secure or cost effective. Due to an increase in [vulnerabilities](#) ↗

found in on-premises VPN products, security and IT teams are looking for solutions that don't require teams to monitor for and respond to [CVE alerts ↗](#). These same systems also limit the user's bandwidth because they route all user Internet traffic through a single infrastructure which results in a poor user experience. IT teams are recognizing the cost and effort to install and maintain their own hardware can be offset with more modern, and more secure cloud hosted services. User expectations for application performance are exposing limitations in bandwidth constrained, self hosted VPN solutions. In summary, running your own VPN is expensive, high risk and doesn't deliver a great user experience.

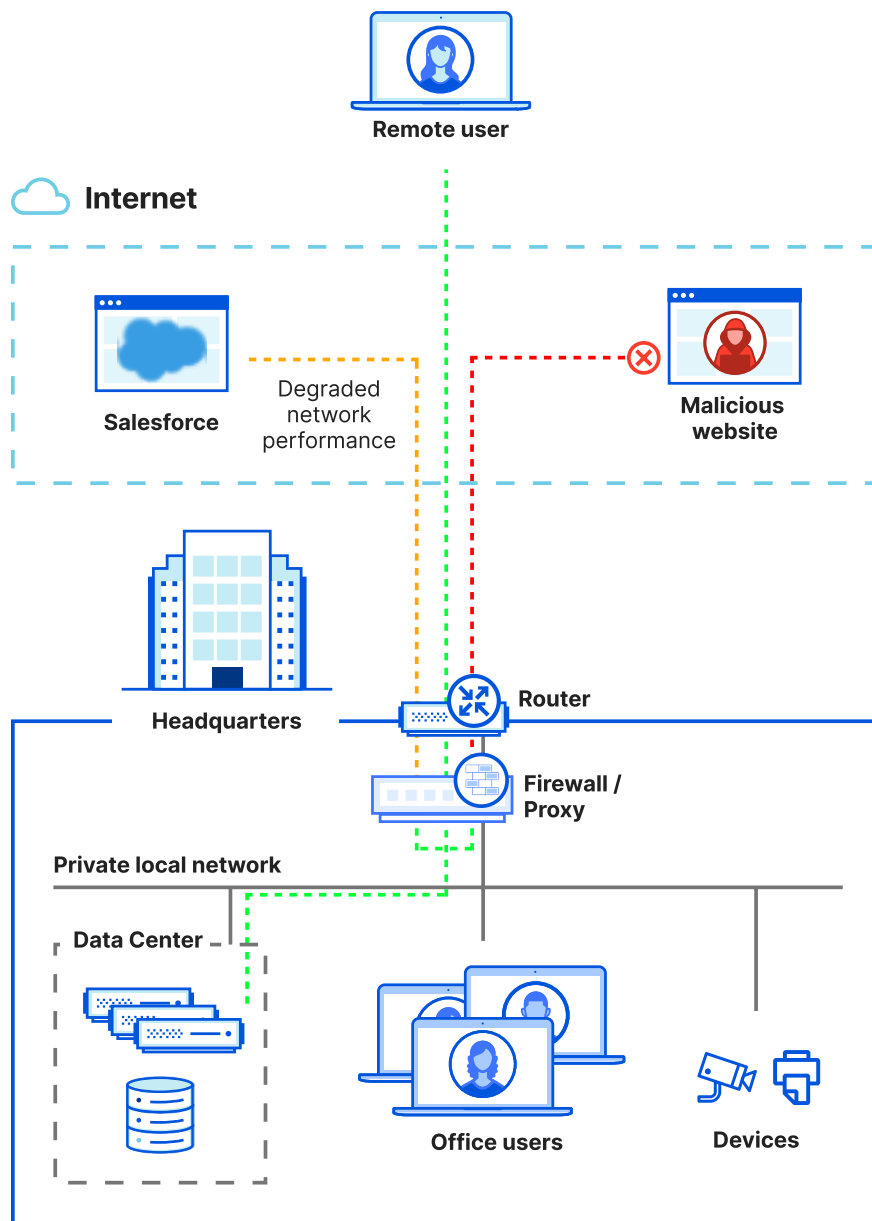


Figure 1: A traditional VPN deployment, where all user traffic destined for the Internet must route through the company hosted and managed VPN service.

As such, many organizations are looking to move to a [zero trust ↗](#) security posture using [Zero Trust Network Access ↗](#) (ZTNA) services as part of a [Secure Access Service Edge ↗](#) (SASE) architecture to provide remote

access to private resources. With all the critical software running as a cloud service, organizations are relieved of the duty of keeping servers and software up to date. Cloud platforms are also architected for massive scale which significantly increases available bandwidth for end users, therefore improving their experience.

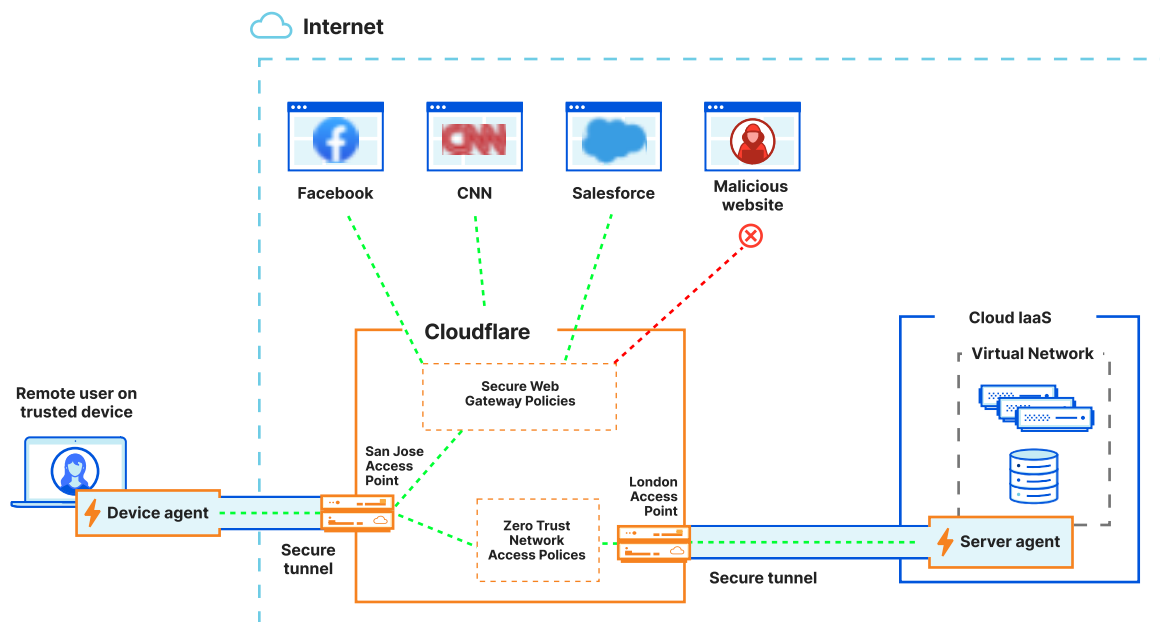


Figure 2: SASE platforms do not degrade user Internet access experience, and provide fast, secure global access to self hosted hosted resources.

In the old model, the VPN hardware had direct access to the networks the applications resided on and

typically users had access to the entire network. New SASE methods of remote access create connectivity from the cloud platform to the networks where applications live, but expose access only to a specific application or network address. Cloudflare's recommended approach is to install software agents, similar to those on end user devices, that create secure tunnels from the cloud to private networks. However, this isn't always an easy path to take. For network administrators trying to quickly replace legacy remote access hardware, having to deploy new servers or go through lengthy change control to deploy software to existing application servers, may not be possible in acceptable time frames. Instead network administrators might be more familiar, and have more control over, creating secure tunnels from cloud SASE platforms to existing network hardware using familiar protocols such as GRE or IPsec. This might even mean using the same hardware appliances that were being used for VPN access, but simply dumbing them down to secure tunnel connectors, and switching off (or removing

licenses for) any expensive and vulnerable remote access capabilities.

This design guide is for organizations in that situation, where they need a fast way to quickly replace or mitigate their use of self hosted remote access hardware and then gradually move to the recommended software agent approach where appropriate.

Audience for this guide

This guide is specifically aimed at network architects or IT admins who want to use familiar protocols and leverage existing network hardware, potentially the same equipment used for current VPN services, but wish to use those devices as tunnel termination devices and move the VPN and access controls into the cloud as part of a longer term migration away from managing their own hardware.

Who is this document for and what will you learn?

This guide is written for network and security experts considering a replacement of their current VPN vendor, while preparing their organization for a zero trust or SASE architecture. It assumes familiarity with networking concepts such as IPsec tunnels, routing tables and split tunneling.

What you will learn:

- How Cloudflare can replace a traditional VPN-like implementation
- How to get visibility into VPN network traffic
- What you need to consider to implement a Cloudflare solution at scale
- Steps to take to move to a recommended Zero Trust Network Access implementation

The solution this guide describes requires you have a contract with Cloudflare that includes:

- Cloudflare One licenses for the amount of users you are looking to onboard
- Cloudflare Magic WAN

To build a stronger baseline understanding of Cloudflare, we recommend the following resources:

1. What is Cloudflare? | [Website ↗](#) (five-minute read) or [video ↗](#) (two minutes)
2. Blog: [What is SASE? | Secure access service edge | Cloudflare ↗](#) (14-minute read)
3. Reference architecture: [Evolving to a SASE architecture with Cloudflare](#) (three-hour read)

Benefits of a SASE platform

Traditional VPN approaches typically provide the following types of access.

- Allowing remote users access to self hosted private applications running on a corporate network

- Routing all user Internet traffic through a single, concentrated VPN access point where security policies are applied

A SASE platform replaces traditional VPN hardware by offering two key services. First, it maps user access directly to internal applications hosted on corporate networks or in the cloud, unlike hosting your own VPN service which typically provides broad access to the entire corporate network. Second, it enables filtering of Internet traffic close to the user, allowing users to securely access the Internet without routing all traffic through the corporate network, thereby improving efficiency and maintaining security.

Zero Trust Network Access (ZTNA)

Remote users authenticate and connect to a cloud hosted Zero Trust Network Access (ZTNA) service, which in turn has connectivity into the networks where the private applications reside. Cloudflare's [SASE](#)

[reference architecture](#) describes three methods for connecting Cloudflare to your existing applications and networks:

1. Software connectors ([cloudflared](#) or [WARP Connector](#))
2. IPsec or GRE tunnels using [Magic WAN](#)
3. Direct network connections using [Cloudflare Network Interconnect](#)

All three methods have their specific advantages, however, software connectors are usually preferred when considering a modern Zero Trust implementation for three reasons.

1. They deliver a network connectivity model that is flexible and easy to replicate across environments. You can move the applications and servers with little to no changes in configuration.
2. Software daemon architecture simplifies scaling to increased traffic demands, just install more agents on more servers.

3. Because daemons run close to your applications (as opposed to at your network edge), you can build isolated network or application segments in which to enforce policy, preventing lateral movement and getting the full benefits of the zero trust model.

Note

This guide will initially describe the use of Magic WAN to create IPsec tunnels from Cloudflare to existing network hardware, and then recommend a migration path to move to a software agent based approach.

Secure Web Gateway (SWG)

Traffic destined for the general Internet is routed via a cloud Secure Web Gateway (SWG). Policies are written that filter requests to malicious websites and allow access to SaaS applications based on user identity and device security posture.

Cloudflare's [SASE reference architecture](#) describes different methods for connecting user devices to Cloudflare, some require the installation of device agents, others require the user simply point their web browser at a URL. In this document, because most traditional VPNs require some client software on the device, we will describe a solution using the Cloudflare [device agent](#).

Why a phased approach?

In situations where existing remote access hardware is vulnerable and there is an urgent need to replace, speed is key. Also, the team tasked with moving away from existing VPN hardware might be more familiar with networks than installing software on servers. Trying to implement a full project to replace existing hardware with a radically different model, that is, deploying software agents, may take weeks rather than days. This guide walks through quickly removing or mitigating existing VPN solutions and then proposes

later steps to take full advantage of using all aspects of a SASE platform.

This approach allows network and security teams to get up-and-running quickly, while gaining experience in modern zero trust deployments to allow for remote access to internal applications. The added visibility into network traffic will also enable teams to gain insight into application usage, and plan for a successful and secure zero trust migration.

This guide will describe the following phases at a high level, if you need help with specific details related to your environment please [contact Cloudflare ↗](#).

- Phase 1: Quickly replace existing traditional/vulnerable VPN hardware with cloud-based remote access while gaining insight into application traffic.
- Phase 2: Scaling up and offloading traditional IPsec tunnels.
- Phase 3: Improving security posture by segmenting application access and enabling

clientless access.

Phase 1: Connectivity and network-based policies

Consider an organization with global IT infrastructure. Specifically, three data centers deployed in Europe, USA and Asia with each their own VPN service. To get the best performance, this VPN implementation requires employees to make a conscious decision to connect to one of the VPN clusters depending on their location. In this example all user Internet traffic is routed through the VPN service, where firewalls apply a level of security protecting users from the dangers of the general Internet.

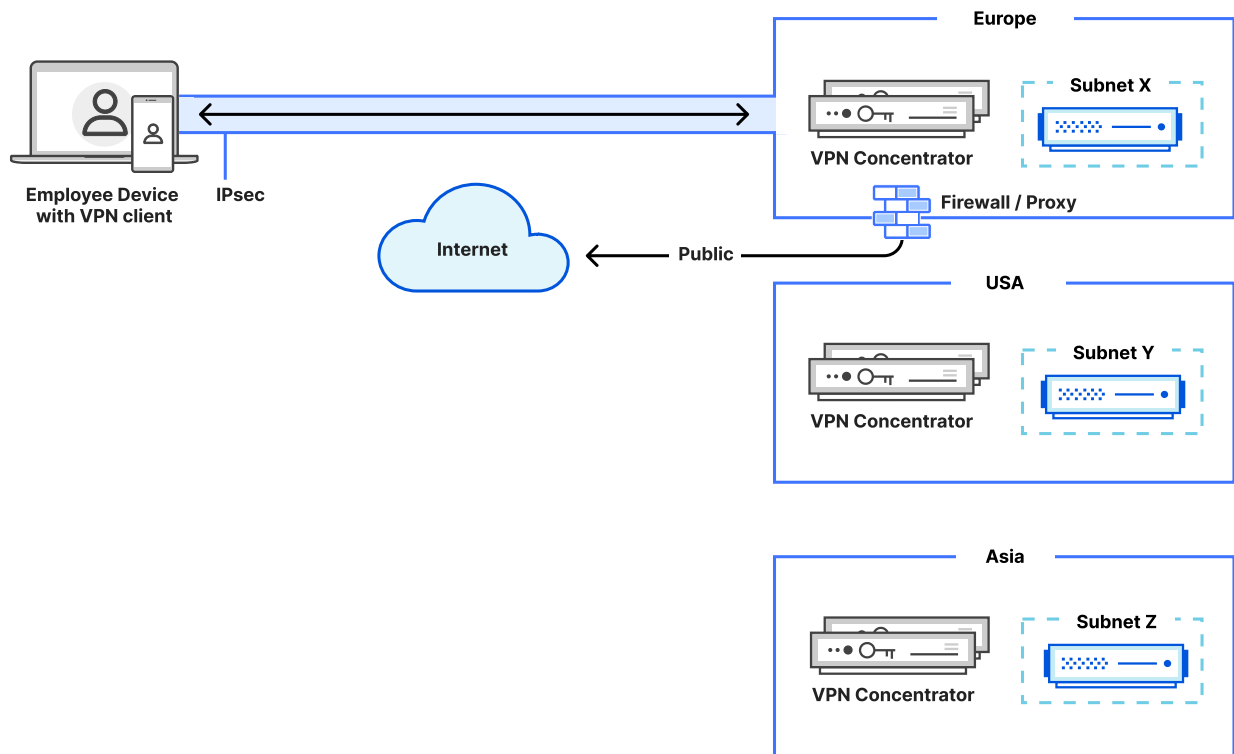


Figure 3: A traditional VPN deployment using VPN concentrators spread across three DCs.

During this first phase, network connectivity will be created between user devices and the private networks they currently access via existing network infrastructure. This is achieved in two ways.

- On employee devices install the Cloudflare [device agent](#). This replaces the use of existing VPN client software.
- Using existing network hardware in the data center, create IPsec tunnels to Cloudflare which

are managed using Cloudflare Magic WAN service.

Both employee devices and data center networks will connect to their closest Cloudflare server. This is thanks to [Cloudflare's anycast architecture ↗](#), and ensures the most optimal path for user traffic without any effort by employees or IT support staff. Users no longer need to make a choice to which VPN service region to connect to, as Cloudflare will always ensure they connect to the closest and most responsive service for the best access performance to their private applications.

Connecting networks to Cloudflare

Figure 4 shows traffic from end user devices to Cloudflare and tunnels routing traffic to private data centers. When user traffic reaches the closest Cloudflare access point, Cloudflare will route traffic destined for private applications directly to the data centers, while processing Internet-bound traffic

through Cloudflare's [Secure Web Gateway](#) (SWG). It is possible to leverage existing DNS services to resolve requests to private addresses using Cloudflare [Gateway DNS policies](#). [Magic WAN](#) is used to create IPsec tunnels between Cloudflare and data centers and is configured with static routes that determine how traffic reaches each existing network and applications.

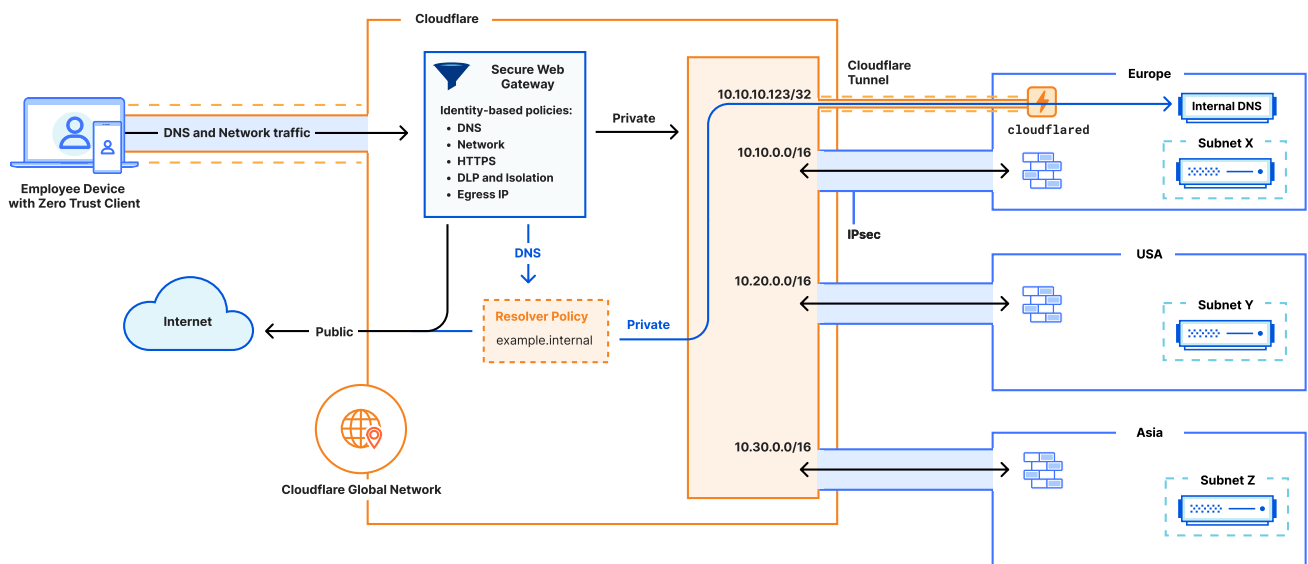


Figure 4: A high level design of Cloudflare traffic routing for phase 1 of the migration.

By using existing network or security appliances to terminate IPsec tunnels, secure off-ramps can be created with limited impact on the current infrastructure. These IPsec tunnels also allow for

outbound server-initiated traffic to continue flowing. However, depending on the scale of the deployment, the existing appliances might run into bandwidth limitations. It is best to consider this first phase a 'pilot' or low-scale deployment to get up and running quickly and validate user-application connectivity. The next phase will improve on the design using the insights gathered during this phase.

With such a design in place, Cloudflare will be able to filter traffic based on the identity of the requesting user. For example, users authenticated to the corporate identity provider and are members of the "Engineering" group will only be allowed access to the internally hosted source code repository. Furthermore, the user device may need to pass [certain posture checks](#) before connecting. There are [example network policies](#) in the zero trust documentation you can use as a reference. In essence, this will enable you to define network access policies using user identities instead of their associated IP address ranges. Getting rid of

traditional 5-tuple ACLs will be a first step towards a zero trust model.

Device agent deployment

Now that we've connected your networks to Cloudflare, we need to get traffic from employee devices to the Cloudflare network which requires the [device agent](#). When the agent is initially installed, users are prompted to authenticate via an identity provider (IdP) configured with Cloudflare. The IdP will ensure users authenticate using an existing identity and can also import group membership information used in access policies. [Device enrollment policies](#) are used to ensure only the right users, authenticated with the right methods and using secure devices can connect new devices to your organization's Cloudflare Zero Trust instance before they even get access to any applications.

Use [device profiles](#) to apply different device agent configurations to different users – or the same users in

different locations using [Managed networks](#). For companies which don't route Internet traffic via their VPN server, device profiles allow you to [configure the device agent to exclude Internet traffic](#) from the Cloudflare tunnel and connect directly to the Internet. Note that this guide does heavily recommend sending Internet bound traffic via Cloudflare where you have greater control over the security of that traffic. But you can selectively bypass Cloudflare for bandwidth heavy traffic such as video conference calls.

Traffic from employees using the device agent destined for internal resources will have a source IP in the 100.96.0.0/12 IP range. This is a range from the [RFC 6598 Carrier-grade NAT space](#) [↗](#) which should be added as a route in the data center regions to allow for traffic to flow back to these users. See for more information the [Magic WAN with WARP integration](#) documentation.

Deploying software connectors for DNS

Although this phase focuses on using the Magic WAN service and IPsec tunnels for the bulk of the employee traffic, the Cloudflare software connectors play a key role in DNS resolution of internal hostnames. Getting experience with using these software connectors will also help in the next phase, so efforts to define the processes to deploy and manage them should start in this first phase.

Cloudflare offers two types of software connectors:

- [ccloudflared](#)
- [WARP connector](#)

As discussed in the introduction, `ccloudflared` is the preferred method for Zero Trust Network Access, but only supports inbound connectivity to your networks and application servers, any server initiated connection will not go via the tunnel and instead follow the server's default network path. WARP connector is designed to create tunnels that facilitate both inbound and outbound connectivity, but it doesn't currently have the same level of failover support and ease of

configuration. For this guide, we will be discussing using `ccloudflared` as it supports the internal DNS use case described.

For large remote access use cases, Cloudflare recommends deploying connectors to dedicated hosts. See the [System Requirements documentation](#) for more deployment recommendations and server sizing.

Where to deploy these servers depends on the access they need and the internal firewall rules and segmentation of the network. Some customers start with their first deployment in their DMZ, while others install it deeper in their network and evolve from there.

Installing `ccloudflared` is best done in an automated manner, so we recommend deploying using a virtualization technology such as Docker or deploying as VMware guests and configuring via Ansible.

Preferably, as traffic using `ccloudflared` tunnels increases, such systems can scale the deployment automatically based on real-time metrics collected from the hosts. `ccloudflared` instances can be

monitored using the [Prometheus metrics endpoint](#).

Prometheus is an HTTP-based monitoring and alerting system similar in functionality to SNMP, exposing metrics that can be polled from the resource to be monitored. Most monitoring systems on the market today support Prometheus as a format to collect the metrics needed for alerting and automatically scaling the deployment.

For more information about deploying ccloudflared connectors at scale:

- [Various guides to deploy and update](#) connectors in environments such as Ansible, Terraform and Kubernetes
- High availability using [replicas](#)
- [Monitor tunnels with Grafana](#)

DNS resolution with Resolver Policies

As you can see in Figure 4, both DNS and general network traffic will flow from the employee device to Cloudflare. By default, the device agent forwards all DNS queries to Cloudflare for inspection and filtering based on [DNS policies](#). This is great, because it will allow administrators to configure [DNS policies to block potential security threats](#) and immediately start to protect employees as they go online. This also applies to situations where Internet traffic is from the tunnel to Cloudflare, but the client still resolves hostname requests via Cloudflare DNS services.

For internal domains, however, Cloudflare will need to know how to resolve them. This is where [resolver policies](#) come into play. After the DNS policies are applied to incoming DNS requests, customers can choose to forward requests for internal DNS hostnames to their internal DNS servers. For example, the domain `example.local` might be hosted on a DNS server running at `10.10.10.123`. A resolver policy will make sure requests for hostnames part of that domain will be sent to that IP.

A tunnel exposing a route to the internal DNS server is needed. `cloudflared` should be deployed on a host that can route DNS traffic to the 10.10.10.123 IP address. Requests for internal domains via the DNS gateway will then be redirected to this DNS server, via the tunnel.

Analytics and logging

As steps are taken in this first phase and the first users will start accessing applications, the need for proper monitoring and logging will become apparent. Having visibility into the traffic flowing through Cloudflare will help with:

- Operational activities such as troubleshooting by your support staff.
- Monitoring for potential threats by a SOC, possibly using a security information and event management ([SIEM ↗](#)) service.
- Visibility into application traffic to see where potential security and performance

improvements can be made (see also phase 2).

Cloudflare provides visibility at different levels, available through the dashboard or exported using [Logpush](#). For traffic flowing over Magic WAN IPsec tunnels, [Network Analytics](#) can be found in the dashboard and through the [GraphQL API](#). This will show sampled statistics of the traffic and can be used for trend and traffic flow analysis.

Next are more detailed [network session logs](#) that collect information on all network connections/sessions going through Cloudflare's secure web gateway, including unsuccessful requests. These are followed by [Gateway activity logs](#), which contain information about triggered policies as traffic gets inspected by the gateway engine. A combination of these logs will enable full visibility into all network flows, including users' identities. Using this information, network and security teams can run their analysis on what type of traffic flows where, and use that to plan for the next steps.

Finally, for real-time alerting, [Cloudflare Notifications](#) can be configured for events such as IPsec and cLOUDFLARED tunnel health, as well as Cloudflare infrastructure status in general.

Phase 2: Scaling up and offloading IPsec

In most environments the IPsec termination points are limited in their throughput and sooner or later this could pose a problem when scaling up to the traffic across the entire business. The final step of phase 1 will provide you with insight into application traffic flow. Although you might not have been able to completely map your application landscape, you probably will have found some applications that cause significant load on the current IPsec tunnels.

Fortunately, most of these applications can be migrated one-by-one to the more scalable software connector based tunnels. Any application which

doesn't rely on server-initiated traffic is eligible for this type of migration. With the experience gained during the initial deployment of `ccloudflared` in phase 1:

1. Deploy two or more `ccloudflared` instances in the relevant environment, the USA datacenter in the example below.
2. Add [Private Networks to the tunnel](#) to define routing and access that is scoped more specifically to the network and applications it handles traffic for. For example, expose the `10.20.56.0/24` subnet via the software connector tunnel, instead of the larger `10.20.0.0/16` exposed by the Magic WAN managed IPsec tunnel.
3. Traffic from employees will now be routed via the software connector tunnel for the `/24` subnet instead of the `/16` route going over the IPsec tunnel, thereby offloading the reliance on the IPsec termination device.

Phase 3: Application-based policies

The first two phases of this guide have resulted in a design very similar to traditional VPNs leveraging VPN concentrators, where policy enforcement happens at the perimeter. Although we've done so for reasons laid out in the introduction, the promise of a zero trust architecture is to improve security posture by defining smaller application/network segments for which security policies are applied as close to the resource as possible.

This phase is about making the resources exposed behind the tunnels smaller and more isolated to prevent lateral movement within internal networks. You will be able to use the visibility gained in the previous phases to select an application (or set of applications), associated IP addresses and deploy a dedicated software connector instance. See [the documentation on how to deploy connectors and expose private](#)

[networks](#) and in step 3 configure the IP addresses of the application.

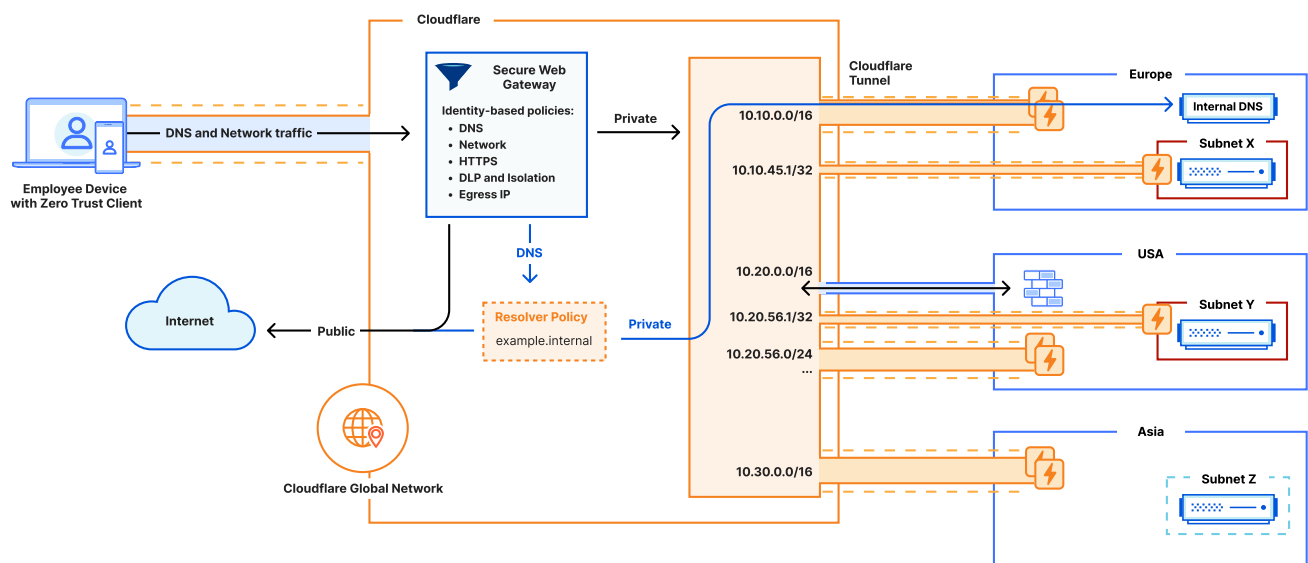


Figure 6: Example phase 3 architecture of tunnels deployed per application to improve security posture by reducing lateral movement within data centers.

Because each software connector instance will be dedicated to the application, it can be configured as the sole entry point. Traffic to and from the network segment where the application resides can be fully blocked off, preventing any internal lateral movement. All that is required is a valid outbound route to the Internet for the software connector to create the tunnel, and for the network/application to be able to

reach the server the software connector is deployed on. The access controls doesn't just manage IP routing, but also at the protocol level. So with this approach you can define access only to HTTPS on that server, which may also be running SSH and other services. But you only want to define access specifically to that application port.

In the example above, subnets X and Y are completely segmented from the rest of the data center. Traffic to the applications running in those subnets (10.10.45.1 and 10.20.56.1, respectively) can only flow through Cloudflare with the associated authentication and authorization policies applied. One to one deployment of software connectors is not always the right approach. You might have several applications running on a private network, and deploy multiple servers running `cloudflared` to handle traffic for the applications.

Clientless access

In addition to routing traffic for private IP addresses, `cloudflared` can expose internal applications via publicly resolvable hostnames. This makes it possible to connect to such applications without using any software on the device. This can be very useful for use cases where you are unable to install software on the device, such as giving application access to contractors or partners.

In the example below, `erp.example.com` is added as [Public Hostname](#) to the tunnel, routing traffic to port 80 and/or 443 to a specific IP address on the internal subnet Y. Access to this resource from the Internet is then protected using [Cloudflare Access security policies](#) which also rely on the IdP connection you've set up for onboarding your employees.

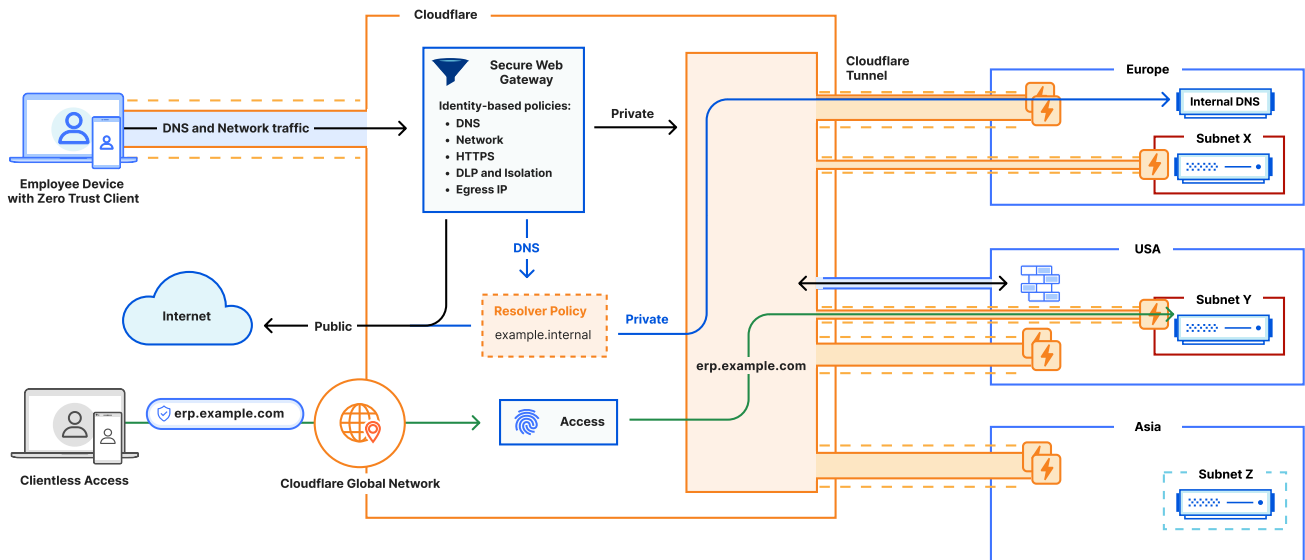


Figure 7: Adding a public hostname to a tunnel for clientless access to internal applications.

Not all applications will be suitable for this type of access. Only HTTP(S) applications or [applications that can be rendered in the browser](#) such as SSH and VNC are supported. To learn more about such a deployment and additional advanced options such cookie settings, browser isolation and using the Access token in your application for authentication, see the [self-hosted application documentation](#).

Summary

This design guide started out with a fairly traditional VPN environment with its common features, limitations and risks. By using a combination of Cloudflare on user devices and Magic WAN towards the datacenter networks, phase one and two described a low-risk design to migrate using existing technology and knowledge. This already brought about benefits in terms of decommissioning of VPN concentrators, improved network visibility and improving performance for users to access internal resources.

Phase three improved on the design by introducing identity-based network policies and smaller network segments with software connectors. This has further opened up the opportunity to offer other zero trust access models such as clientless access for web applications and browser-rendered VNC or SSH sessions.

The flexibility of the Cloudflare connectivity cloud to connect any device, application and network enables this zero trust migration to be taken step by step.

Thereby reducing risk and allowing network and security teams to adapt their knowledge and architectures in the pace required by their organizations.

Further reading

- Magic WAN integration: [WARP on-ramp to Magic WAN](#)
- Policy configuration: [Gateway Network policies](#)

Was this helpful?



Previous

Next

← Designing ZTNA access policies

Secure application delivery →

Last updated: Oct 9, 2025

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

[Open Source](#)

Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)

Community

 [X](#)

 [Discord](#)

 [YouTube](#)

 [GitHub](#)

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

- [Report Security Issues](#)
- [Trademark](#)
-  [Cookie Preferences](#)